

LAPORAN PRATIUM PEMROGRAMAN ALGORITMA DAN PEMROGRAMAN
“DOUBLE LINKED LIST”



disusun Oleh:

Nama : Alhamd Ridho Putra

NIM : 2511532020

Dosen Pengampu : Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum Struktur Data mengenai *Double Linked List* (DLL) dengan baik. Laporan ini disusun guna memenuhi salah satu tugas akademis pada mata kuliah Struktur Data sekaligus mendokumentasikan pemahaman praktis yang diperoleh selama sesi laboratorium.

Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan, arahan, dan dukungan berbagai pihak, baik dosen pengampu maupun rekan-rekan mahasiswa yang saling bertukar pikiran selama pengerjaan kode program dalam ekosistem Java. Penulis mengucapkan terima kasih yang sebesar-besarnya atas segala bantuan yang telah diberikan.

Akhir kata, penulis menyadari bahwa laporan praktikum ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat nyata, menambah wawasan, serta menjadi referensi yang berguna bagi pembaca dalam memahami konsep struktur data dinamis, khususnya implementasi senarai berantai ganda.

Padang, 30 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR

DAFTAR ISI

BAB I PENDAHULUAN	
1.1. Latar Belakang	
1.2. Tujuan Praktikum	
1.3. Manfaat Praktikum.....	
BAB II PEMBAHASAN	
2.1. Uraian Masalah	
2.2. Langkah Kerja.....	
2.3. Analisis Hasil.....	
BAB III KESIMPULAN	
3.1. Kesimpulan	
3.2. Saran Pengembangan.....	
DAFTAR PUSTAKA.....	

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, pengelolaan data di dalam memori secara efisien merupakan salah satu fokus utama pengembangan perangkat lunak. Struktur data linear seperti *array* (larik) statis sering kali menghadapi kendala keterbatasan fleksibilitas alokasi ukuran memori karena penentuan kapasitasnya harus dideklarasikan di awal dan bersifat kontinu [1].

Guna mengatasi keterbatasan alokasi statis tersebut, konsep senarai berantai atau *linked list* dikembangkan sebagai alternatif struktur data dinamis yang mampu mengalokasikan dan membebaskan ruang memori kapan saja secara run-time.

[Salah satu variasi senarai berantai yang paling fleksibel adalah *Double Linked List* (DLL). Berbeda dengan *Single Linked List* yang elemennya hanya memiliki satu referensi penunjuk ke node berikutnya, setiap node dalam DLL dibekali dengan dua referensi penunjuk sekaligus, yaitu penunjuk maju (*next pointer*) dan penunjuk mundur (*prev pointer*) [2]. Karakteristik navigasi dua arah ini memberikan keuntungan komputasi yang signifikan, di mana proses penelusuran (*traversal*) serta operasi manipulasi elemen seperti penyisipan dan penghapusan dapat dilakukan secara efisien dari kedua ujung senarai tanpa harus melakukan penelusuran ulang dari simpul awal [3]. Melalui praktikum ini, implementasi arsitektur DLL dieksplorasi secara mendalam menggunakan bahasa pemrograman Java dengan menerapkan identifikasi nomor induk mahasiswa (NIM) 2511532020 pada variabel dan kelas untuk merepresentasikan keaslian pengerjaan.

1.2. Tujuan Praktikum

Adapun tujuan dilaksanakannya praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar arsitektur *Double Linked List* serta perbedaannya dengan struktur data linear lainnya.
2. Mengimplementasikan struktur node yang memiliki dua buah penunjuk arah (*next* dan *prev*) di dalam bahasa pemrograman Java secara mandiri.
3. Menguasai logika dan algoritma manipulasi struktur data DLL yang meliputi operasi penyisipan elemen (di awal, di akhir, dan posisi tertentu), operasi penghapusan elemen (di awal, di akhir, dan posisi tertentu), serta metode penelusuran dua arah (*forward and backward traversal*).
4. Mampu menganalisis kompleksitas waktu (*Time Complexity*) dan perilaku memori dari setiap fungsi manipulasi yang dijalankan pada program objek.

1.3. Manfaat Praktikum

Praktikum ini diharapkan mampu memberikan manfaat ilmiah maupun praktis kepada praktikan:

- **Manfaat Teoritis:** Memperkuat landasan pemahaman mengenai manajemen memori dinamis dan struktur alokasi pointer di dalam konsep ilmu komputer dasar.
- **Manfaat Praktis:** Mengasah kemampuan logika *problem-solving* dalam menangani kondisi-kondisi batas (*edge cases*) seperti *NullPointerException* pada manipulasi pointer penunjuk, yang sangat berguna dalam pengembangan sistem perangkat lunak riil seperti fitur *undo/redo* maupun

sistem navigasi halaman web.

BAB II

PEMBAHASAN

2.1. Uraian Masalah

Pada praktikum ini, implementasi program dibagi ke dalam empat unit kelas utama yang saling berintegrasi dalam paket bernama `pekan6_2511532020`. Berikut adalah pemaparan dan analisis rinci terhadap arsitektur masing-masing blok kode:

1. Kode Kelas Struktur Node (NodeDLL_2511532020.java)

Kelas ini merupakan fondasi dasar pembentuk objek individu di dalam struktur data senarai berantai ganda. Struktur internal dari kelas ini dideklarasikan sebagai berikut:

Analisis Struktur Node: Setiap instansiasi dari kelas ini melahirkan sebuah entitas memori yang memuat tiga komponen esensial: variabel integer `data_2020` sebagai penampung nilai riil, serta dua variabel objek bertipe sama yaitu `next_2020` (menyimpan alamat memori simpul sesudahnya) dan `prev_2020` (menyimpan alamat memori simpul sebelumnya). Konstruktor kelas memastikan bahwa pada saat objek node baru dilahirkan, kedua variabel penunjuk secara bawaan diarahkan ke nilai kosong (`null`) untuk mencegah inkonsistensi referensi data pointer [4].

```
package pekan6_2511532020;
```

```

public class NodeDLL_2511532020 { int
    data_2020; // data

    NodeDLL_2511532020 next_2020;
    NodeDLL_2511532020 prev_2020;

    public NodeDLL_2511532020(int data_2020) {
        this.data_2020 = data_2020;
        this.next_2020 = null; this.prev_2020
        = null;
    }
}

```

2. Kode Kelas Operasi Penyisipan (InsertDLL_2511532020.java)

Kelas ini mengemas tiga skenario algoritma penyisipan node ke dalam sistem senarai berantai ganda, ditambah satu fungsi pembantu untuk menampilkan seluruh elemen list.

```

package pekan6_2511532020;

public class InsertDLL_2511532020 {

    static NodeDLL_2511532020 insertBegin_2020(NodeDLL_2511532020
head_2020, int data_2020) {

        NodeDLL_2511532020 new_node_2020 = new
NodeDLL_2511532020(data_2020);

        new_node_2020.next_2020 = head_2020;
        if (head_2020 != null) {

            head_2020.prev_2020 = new_node_2020;

        }

        return new_node_2020;

    }

    public static NodeDLL_2511532020 insertEnd_2020(NodeDLL_2511532020
head_2020, int newData_2020) {

        NodeDLL_2511532020 newNode_2020 = new
NodeDLL_2511532020(newData_2020);

        if (head_2020 == null) {

            head_2020 = newNode_2020;

        }

    }
}

```

```

else {

    NodeDLL_2511532020 curr_2020 = head_2020;
    while (curr_2020.next_2020 != null) {

        curr_2020 = curr_2020.next_2020;

    }

    curr_2020.next_2020 = newNode_2020;
    newNode_2020.prev_2020 = curr_2020;

}

return head_2020;

}

public static NodeDLL_2511532020 insertAtPosition_2020(NodeDLL_2511532020
head_2020, int pos_2020, int new_data_2020) {

    NodeDLL_2511532020 new_node_2020 = new
NodeDLL_2511532020(new_data_2020);

    if (pos_2020 == 1) {
        new_node_2020.next_2020 = head_2020;
        if (head_2020 != null) {

            head_2020.prev_2020 = new_node_2020;

        }

        head_2020 = new_node_2020;
        return head_2020;

    }

    NodeDLL_2511532020 curr_2020 = head_2020;

    for (int i = 1; i < pos_2020 - 1 && curr_2020 != null; i++) {
        curr_2020 = curr_2020.next_2020;

    }

    if (curr_2020 == null) {
        System.out.println("Posisi Tidak Ada");
        return head_2020;

    }

    new_node_2020.prev_2020 = curr_2020;
    new_node_2020.next_2020 = curr_2020.next_2020;
    curr_2020.next_2020 = new_node_2020;

    if (new_node_2020.next_2020 != null) {
        new_node_2020.next_2020.prev_2020 = new_node_2020;
    }
}

```

```

    }

    return head_2020;
}

public static void printList_2020(NodeDLL_2511532020 head_2020) {
    NodeDLL_2511532020 curr_2020 = head_2020;

    while (curr_2020 != null) {
        System.out.print(curr_2020.data_2020 + " <-> ");
        curr_2020 = curr_2020.next_2020;
    }

    System.out.println();
}

public static void main(String[] args) {
    NodeDLL_2511532020 head_2020 = new NodeDLL_2511532020(2);
    head_2020.next_2020 = new NodeDLL_2511532020(3);
    head_2020.next_2020.prev_2020 = head_2020;
    head_2020.next_2020.next_2020 = new NodeDLL_2511532020(5);
    head_2020.next_2020.next_2020.prev_2020 = head_2020.next_2020;

    System.out.print("DLL Awal : ");
    printList_2020(head_2020);

    head_2020 = insertBegin_2020(head_2020, 1);
    System.out.print("simpul 1 ditambah di awal: ");
    printList_2020(head_2020);
    System.out.print("simpul 6 ditambah di akhir: ");
    int data_2020 = 6;

    head_2020 = insertEnd_2020(head_2020, data_2020);
    printList_2020(head_2020);

    System.out.print("tambah node 4 di posisi 4: "); int
    data2_2020 = 4;

    int pos_2020 = 4;

    head_2020 = insertAtPosition_2020(head_2020, pos_2020, data2_2020);
    printList_2020(head_2020);
}

```

Analisis Operasi Penyisipan: Fungsi `insertBegin_2020` menempatkan data baru di hulu senarai. Kompleksitas waktunya adalah $O(1)$ karena tidak membutuhkan perulangan. Fungsi

insertEnd_2020 digunakan untuk penyisipan di hilir senarai; apabila list tidak kosong, dilakukan penelusuran linier hingga elemen ujung menggunakan penanda komputasi loop while (curr_2020.next_2020 != null), menghasilkan waktu operasi sebesar $O(N)$. Adapun metode insertAtPosition_2020 mengakomodasi penyisipan data pada koordinat indeks tertentu dengan melakukan perulangan bertahap (*for-loop*) guna mengamankan simpul acuan sebelum merestrukturisasi empat jalur link penghubung pointer [5].

3. Kode Kelas Mekanisme Penelusuran (PenelusuranDLL_2511532020.java)

Kelas ini mendemonstrasikan keunggulan utama dari struktur data DLL, yakni kapabilitas penelusuran simpul secara bolak-balik dari arah depan maupun belakang.

```
package pekan6_2511532020;

public class PenelusuranDLL_2511532020 {

    static void forwardTraversal_2020(NodeDLL_2511532020 head_2020) {
        NodeDLL_2511532020 curr_2020 = head_2020;

        while (curr_2020 != null) {

            System.out.print(curr_2020.data_2020 + " <-> ");
            curr_2020 = curr_2020.next_2020;

        }

        System.out.println();
    }

    static void backwardTraversal_2020(NodeDLL_2511532020 tail_2020) {
        NodeDLL_2511532020 curr_2020 = tail_2020;

        while (curr_2020 != null) {

            System.out.print(curr_2020.data_2020 + " <-> ");
            curr_2020 = curr_2020.prev_2020;

        }

        System.out.println();
    }

    public static void main(String[] args) {

        NodeDLL_2511532020 head_2020 = new NodeDLL_2511532020(1);
        NodeDLL_2511532020 second_2020 = new NodeDLL_2511532020(2);
        NodeDLL_2511532020 third_2020 = new NodeDLL_2511532020(3);

        head_2020.next_2020 = second_2020;
        second_2020.prev_2020 = head_2020;
        second_2020.next_2020 = third_2020;
        third_2020.prev_2020 = second_2020;
    }
}
```

```

        System.out.println("Penelusuran maju: ");
        forwardTraversal_2020(head_2020);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal_2020(third_2020);
    }
}

```

Analisis Penelusuran: Fungsi `forwardTraversal_2020` melangkah ke kanan dengan memanfaatkan pembaruan iterasi `curr_2020 = curr_2020.next_2020`, sedangkan fungsi `backwardTraversal_2020` melangkah berlawanan arah memanfaatkan link balik `curr_2020 = curr_2020.prev_2020` bermodalkan node ekor (tail atau `third_2020`). Kedua fungsi memiliki kompleksitas waktu linier $O(N)$ [3].

4. Kode Kelas Operasi Penghapusan (HapusDLL_2511532020.java)

Kelas ini bertanggung jawab memutus hubungan objek node di dalam senarai berantai, mereduksi ukuran struktur data secara dinamis.

```

package pekan6_2511532020;

public class HapusDLL_2511532020 {
    public static NodeDLL_2511532020 delHead_2020(NodeDLL_2511532020
head_2020) {
        if (head_2020 == null) {
            return null;
        }
        head_2020 = head_2020.next_2020;
        if (head_2020 != null) {
            head_2020.prev_2020 = null;
        }
        return head_2020;
    }

    public static NodeDLL_2511532020 delLast_2020(NodeDLL_2511532020
head_2020) {
        if (head_2020 == null) {
            return null;
        }
    }
}

```

```

        if (head_2020.next_2020 == null) {
            return null;
        }

        NodeDLL_2511532020 curr_2020 = head_2020;
        while (curr_2020.next_2020 != null) {

            curr_2020 = curr_2020.next_2020;

        }

        if (curr_2020.prev_2020 != null) {
            curr_2020.prev_2020.next_2020 = null;
        }

        return head_2020;
    }

    public static NodeDLL_2511532020 delPos_2020(NodeDLL_2511532020
head_2020, int pos_2020) {

        if (head_2020 == null) {
            return head_2020;
        }

        NodeDLL_2511532020 curr_2020 = head_2020;

        for (int i = 1; curr_2020 != null && i < pos_2020; i++) {
            curr_2020 = curr_2020.next_2020;
        }

        if (curr_2020 == null) {
            return head_2020;
        }

        if (curr_2020.prev_2020 != null) {
            curr_2020.prev_2020.next_2020 = curr_2020.next_2020;
        }

        if (curr_2020.next_2020 != null) {
            curr_2020.next_2020.prev_2020 = curr_2020.prev_2020;
        }

        if (head_2020 == curr_2020) {
            head_2020 = curr_2020.next_2020;
        }

        return head_2020;
    }
}

```

```

public static void printList_2020(NodeDLL_2511532020 head_2020) {
    NodeDLL_2511532020 curr_2020 = head_2020;

    while (curr_2020 != null) {
        System.out.print(curr_2020.data_2020 + " ");
        curr_2020 = curr_2020.next_2020;
    }

    System.out.println();
}

public static void main(String[] args) {
    NodeDLL_2511532020 head_2020 = new NodeDLL_2511532020(1);
    head_2020.next_2020 = new NodeDLL_2511532020(2);
    head_2020.next_2020.prev_2020 = head_2020;
    head_2020.next_2020.next_2020 = new NodeDLL_2511532020(3);
    head_2020.next_2020.next_2020.prev_2020 =
head_2020.next_2020;

    head_2020.next_2020.next_2020.next_2020 = new
NodeDLL_2511532020(4);

    head_2020.next_2020.next_2020.next_2020.prev_2020 =
head_2020.next_2020.next_2020;

    head_2020.next_2020.next_2020.next_2020.next_2020 = new
NodeDLL_2511532020(5);

    head_2020.next_2020.next_2020.next_2020.next_2020.prev_2020 =
head_2020.next_2020.next_2020.next_2020;

    System.out.print("DLL Awal: ");
    printList_2020(head_2020);
    System.out.print("setelah head dihapus: ");
    head_2020 = delHead_2020(head_2020);
    printList_2020(head_2020);

    System.out.print("setelah node terakhir di hapus: ");
    head_2020 = delLast_2020(head_2020);
    printList_2020(head_2020);
    System.out.print("menghapus node ke 2: ");

    head_2020 = delPos_2020(head_2020, 2);
    printList_2020(head_2020);
}
}

```

Analisis Operasi Penghapusan: Metode delHead_2020 menggeser pointer acuan depan

ke elemen kedua dan memutus ikatan `prev_2020` menjadi `null` dalam ongkos waktu $O(1)$. Fungsi `delLast_2020` menelusuri senarai hingga elemen paling ujung kanan kemudian memutuskan tautan balik dari komponen penunjuk sebelumnya dengan waktu $O(N)$. Operasi `delPos_2020` bekerja secara spesifik mencari node sasaran pada urutan posisi numerik tertentu dan menghubungkan simpul sebelum dan sesudah objek terhapus agar struktur integritas rantai memori tidak terputus [6].

2.2. Langkah Kerja

Operasional pengerjaan praktikum di laboratorium dilaksanakan mengikuti sekuens terstruktur di bawah ini:

1. **Mempersiapkan Lingkungan Kerja:** Memerlukan perangkat lunak Integrated Development Environment (IDE) seperti Eclipse atau IntelliJ IDEA yang mendukung Java Development Kit (JDK) versi 8 ke atas.
2. **Membuat Package Kerja:** Membuat sebuah paket baru bernama `pekan6_2511532020` untuk menampung seluruh berkas kompilasi kelas praktikum ini.
3. **Implementasi Node Dasar:** Menuliskan berkas kelas `NodeDLL_2511532020.java` sesuai rancangan cetak biru node dua arah yang memuat variabel `data_2020`, `next_2020`, dan `prev_2020`.
4. **Implementasi Kode Penunjang:** Mengetikkan dan melengkapi fungsionalitas algoritma pada berkas `InsertDLL_2511532020.java`, `PenelusuranDLL_2511532020.java`, dan `HapusDLL_2511532020.java` secara berurutan.
5. **Eksekusi dan Pengujian:** Menjalankan metode utama (`main method`) pada setiap kelas penyangga untuk mengamati keluaran terminal konsol teks.
6. **Validasi Kondisi Batas:** Melakukan analisis pengetesan alur kode jika parameter input bernilai kosong atau posisi indeks di luar jangkauan untuk memastikan kestabilan program dari *crash*.

2.3. Analisis Hasil

Berdasarkan eksekusi baris perintah utama pada masing-masing modul kode program, didapatkan kompilasi transkrip keluaran (*output text*) yang presisi sebagai berikut:

1. Hasil Kelas Operasi Penyisipan (InsertDLL_2511532020)

```
DLL Awal : 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Analisis Logika Eksekusi: Pada mulanya senarai diinisialisasi secara manual membentuk struktur 2 <-> 3 <-> 5. Pemanggilan metode `insertBegin_2020` dengan argumen data bernilai 1 berhasil memposisikan objek tersebut di pucuk rantai, memperbarui orientasi variabel penunjuk hulu utama. Penyisipan angka 6 di bagian ekor berhasil dieksekusi secara linier melalui fungsi penelusuran akhir. Akhir operasi ditutup oleh metode penyisipan posisi ke-4, di mana kode berhasil menempatkan data 4 di sela-sela angka 3 dan 5 tanpa merusak keutuhan rantai referensi memori penunjuk maju maupun mundur.

2. Hasil Kelas Mekanisme Penelusuran (PenelusuranDLL_2511532020)

```
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Analisis Logika Eksekusi: Hasil keluaran memvalidasi secara mutlak bahwa objek data yang tersusun dapat diakses secara simetris dari dua sudut pandang berbeda. Penelusuran maju (*forward*) mencetak barisan nilai dari kiri ke kanan bermodalkan `head_2020`, sementara penelusuran mundur (*backward*) membalik arah pencetakan dari kanan ke kiri berbekal penunjuk alamat objek `third_2020` selaku simpul ekor (*tail*). Hal ini membuktikan tautan `prev_2020` berfungsi dengan sempurna di dalam alokasi memori internal.

3. Hasil Kelas Operasi Penghapusan (HapusDLL_2511532020)

```
DLL Awal: 1 2 3 4 5
setelah head dihapus: 2 3 4 5
setelah node terakhir di hapus: 2 3 4
menghapus node ke 2: 2 4
```

Analisis Logika Eksekusi: Senarai awal memiliki lima elemen (1 sampai 5). Pasca pemanggilan fungsi `delHead_2020`, angka 1 tereliminasi secara permanen dan acuan list berpindah ke angka 2. Selanjutnya, fungsi `delLast_2020` mengeksekusi pemotongan elemen ekor sehingga angka 5 terbuang dari daftar aktif. Operasi pamungkas adalah pemanggilan

`delPos_2020(head_2020, 2);` dalam urutan daftar tersisa (2 3 4), posisi urutan kedua dihuni oleh data berangka 3. Logika program berhasil menjembatani penunjuk milik angka 2 langsung mengarah ke objek angka 4, memutus interaksi memori terhadap angka 3 sehingga angka 3 terhapus dan menghasilkan keluaran akhir berupa senarai berantai ganda yang ringkas yaitu 2 4 .

BAB III

KESIMPULAN

3.1. Kesimpulan

Melalui hasil pengerjaan, pengujian, dan analisis mendalam pada praktikum ini, dapat ditarik beberapa kesimpulan penting sebagai berikut:

1. *Double Linked List* (DLL) terbukti memberikan tingkat fleksibilitas navigasi yang jauh lebih tinggi dibandingkan dengan *Single Linked List* karena adanya dua buah pointer referensi ganda (`next` dan `prev`) pada setiap simpul penyusunnya [2].
2. Operasi manipulasi data pada DLL seperti penyisipan dan penghapusan di posisi hulu (*head*) memiliki efisiensi waktu konstan **$O(1)$** , namun operasi manipulasi di akhir senarai maupun di posisi spesifik membutuhkan waktu linier **$O(N)$** akibat keharusan melakukan proses penelusuran sekuensial terlebih dahulu [5].
3. Ketelitian dalam mengatur ulang urutan penugasan pointer (*pointer assignment ordering*) sangat krusial pada DLL. Kesalahan urutan pemutusan atau penyambungan tautan referensi akan mengakibatkan hilangnya akses ke sebagian atau seluruh simpul dalam memori, atau memicu galat fatal berupa *NullPointerException*.

3.2. Saran Pengembangan

Untuk menyempurnakan implementasi serta memperluas pemanfaatan struktur data ini di masa mendatang, disarankan beberapa poin pengembangan berikut:

- **Transformasi ke Circular Double Linked List:** Struktur ini dapat dikembangkan lebih lanjut menjadi senarai ganda melingkar, di mana pointer `next` dari node terakhir langsung terhubung ke node pertama, dan pointer `prev` dari node pertama terhubung ke node terakhir demi mengoptimalkan penelusuran tanpa batas ujung [1].
- **Penerapan Java Generics:** Mengganti tipe data primitif `int data_2020` dengan parameter generik `<T>` agar kelas `NodeDLL` mampu menyimpan berbagai macam tipe data objek lain yang lebih kompleks dan heterogen.

- **Enkapsulasi Kelas dan Implementasi Sentinel Node:** Membungkus fungsi-fungsi statis ke dalam satu kelas manajemen senarai objek khusus (kelas `DoubleLinkedList`) serta menggunakan simpul pembantu (*dummy/sentinel node*) di bagian hulu dan hilir untuk menyederhanakan logika pengecekan kondisi list kosong atau list berisi satu elemen [4].

TUGAS PRAKTIKUM

1. KODE

```
package pekan6_2511532020;

//Kelas Node untuk merepresentasikan satu lagu
public class Lagu_2511532020 {
    // Atribut wajib menggunakan akhiran 4 digit NIM
    private String judul_2020;
    private String penyanyi_2020;

    // Pointer next dan prev wajib pakai akhiran NIM
    public Lagu_2511532020 next_2020;
    public Lagu_2511532020 prev_2020;

    // Constructor untuk inisialisasi lagu baru
    public Lagu_2511532020(String judul, String penyanyi) {
        this.judul_2020 = judul;
        this.penyanyi_2020 = penyanyi;
        this.next_2020 = null;
        this.prev_2020 = null;
    }

    // Getter dan Setter untuk judul
    public String getJudul_2020() {
        return judul_2020;
    }

    public void setJudul_2020(String judul_2020) {
        this.judul_2020 = judul_2020;
    }

    // Getter dan Setter untuk penyanyi
    public String getPenyanyi_2020() {
        return penyanyi_2020;
    }

    public void setPenyanyi_2020(String penyanyi_2020) {
        this.penyanyi_2020 = penyanyi_2020;
    }
}
```

2. DRIVER

```
package pekan6_2511532020;

import java.util.Scanner;

public class Musik_2511532020 {
    // Pointer head dan tail untuk menandai awal dan akhir playlist
    private Lagu_2511532020 head_2020 = null;
    private Lagu_2511532020 tail_2020 = null;

    // 1. Menambah lagu baru di AKHIR playlist
    public void tambahLagu_2020(String judul, String penyanyi) {
        Lagu_2511532020 laguBaru = new Lagu_2511532020(judul, penyanyi);

        if (head_2020 == null) {
            // Jika playlist masih kosong
            head_2020 = laguBaru;
            tail_2020 = laguBaru;
        } else {
            // Menghubungkan node baru di akhir (tail)
            tail_2020.next_2020 = laguBaru;
            laguBaru.prev_2020 = tail_2020;
            tail_2020 = laguBaru; // Geser tail ke lagu baru
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }

    // 2. Menghapus lagu pertama (head)
    public void hapusLaguAwal_2020() {
        if (head_2020 == null) {
            System.out.println("Playlist kosong, tidak ada lagu yang bisa dihapus.");
            return;
        }

        System.out.println("Menghapus lagu: " + head_2020.getJudul_2020() + " - " + head_2020.getPenyanyi_2020());

        if (head_2020 == tail_2020) {
            // Jika hanya ada satu lagu di playlist
            head_2020 = null;
            tail_2020 = null;
        } else {
            // Menggeser head ke lagu berikutnya dan memutus hubungan dengan head lama
            head_2020 = head_2020.next_2020;
            head_2020.prev_2020 = null;
        }
    }

    // 3. Menampilkan semua lagu dari awal ke akhir (Maju)
    public void tampilMaju_2020() {
        if (head_2020 == null) {
            System.out.println("Playlist kosong.");
            return;
        }
    }
}
```

```

    }

    System.out.println("\n=== PLAYLIST (MAJU) ===");
    Lagu_2511532020 current = head_2020;
    int nomor = 1;
    while (current != null) {
        System.out.println(nomor + ". " + current.getJudul_2020() +
" (" + current.getPenyanyi_2020() + ")");
        current = current.next_2020;
        nomor++;
    }
}

// 4. Menampilkan semua lagu dari akhir ke awal (Mundur)
public void tampilMundur_2020() {
    if (tail_2020 == null) {
        System.out.println("Playlist kosong.");
        return;
    }

    System.out.println("\n=== PLAYLIST (MUNDUR) ===");
    Lagu_2511532020 current = tail_2020;
    int nomor = 1;
    while (current != null) {
        System.out.println(nomor + ". " + current.getJudul_2020() +
" (" + current.getPenyanyi_2020() + ")");
        current = current.prev_2020; // Berjalan mundur menggunakan
pointer prev
        nomor++;
    }
}

// 5. Mencari lagu berdasarkan judul (tidak case-sensitive)
public void cariLagu_2020(String judul) {
    if (head_2020 == null) {
        System.out.println("Playlist kosong.");
        return;
    }

    Lagu_2511532020 current = head_2020;
    boolean ditemukan = false;

    while (current != null) {
        // Menggunakan equalsIgnoreCase agar pencarian tidak case-
sensitive
        if (current.getJudul_2020().equalsIgnoreCase(judul)) {
            System.out.println("Lagu Ditemukan!");
            System.out.println("Judul    : " +
current.getJudul_2020());
            System.out.println("Penyanyi: " +
current.getPenyanyi_2020());
            ditemukan = true;
            break;
        }
        current = current.next_2020;
    }
}

```

```

        if (!ditemukan) {
            System.out.println("Lagu dengan judul '" + judul + "' tidak
ditemukan.");
        }
    }

    // Main Method untuk menjalankan menu aplikasi
    public static void main(String[] args) {
        Musik_2511532020 playlist = new Musik_2511532020();
        Scanner scanner = new Scanner(System.in);
        int pilihan;

        do {
            System.out.println("\nPlaylist Musik NIM: 2511532020");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan = scanner.nextInt();
            scanner.nextLine(); // Membersihkan buffer

            switch (pilihan) {
                case 1:
                    System.out.print("Judul: ");
                    String judul = scanner.nextLine();
                    System.out.print("Penyanyi: ");
                    String penyanyi = scanner.nextLine();
                    playlist.tambahLagu_2020(judul, penyanyi);
                    break;
                case 2:
                    playlist.hapusLaguAwal_2020();
                    break;
                case 3:
                    playlist.tampilMaju_2020();
                    break;
                case 4:
                    playlist.tampilMundur_2020();
                    break;
                case 5:
                    System.out.print("Masukkan Judul Lagu yang Dicari:
");

                    String cariJudul = scanner.nextLine();
                    playlist.cariLagu_2020(cariJudul);
                    break;
                case 6:
                    System.out.println("Terima kasih!");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 6);

        scanner.close();
    }
}

```

}

DAFTAR PUSTAKA

- [1] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [3] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.
- [4] S. Sahni, *Data Structures, Algorithms, and Applications in Java*, 2nd ed. Silicon Valley, CA, USA: Universities Press, 2005.
- [5] N. Karumanchi, *Data Structures and Algorithms Made Easy: Data Structures and Algorithmic Puzzles*, 5th ed. Hyderabad, India: CareerMonk Publications, 2016.
- [6] Y. D. Liang, *Introduction to Java Programming and Data Structures*, 12th ed. New York, NY, USA: Pearson, 2020.